

# Lovely Professional University, Punjab

| Course Code      | Course Title                         | Lectures | Tutorials | Practicals | Credits |  |
|------------------|--------------------------------------|----------|-----------|------------|---------|--|
| CSE408           | DESIGN AND ANALYSIS OF ALGORITHMS    | 3        | 0         | 2          | 4       |  |
| Course Weightage | ATT: 5   CAP: 25   MTT: 20   ETT: 50 |          |           |            |         |  |
| Course Focus     | EMPLOYABILITY,SKILL DEVELOPMENT      |          |           |            |         |  |

**Course Outcomes :**Through this course students should be able to

CO1 :: Analyze the efficiency of algorithms using complexity analysis, recurrence solving techniques, and Divide-and-Conquer strategies.

CO2 :: Apply string matching algorithms, Trie data structures, and computational geometry techniques to solve pattern matching and geometric problems.

CO3 :: Apply Dynamic Programming techniques to solve optimization and combinatorial problems.

CO4 :: Apply Graph algorithms, Network Optimization, and Greedy techniques to solve graph-based optimization problems.

CO5 :: Apply Transform-and-Conquer and advanced algorithmic techniques to solve searching, sorting, and sequence-processing problems efficiently.

CO6 :: Evaluate the suitability of Backtracking, Branch and Bound, Approximation Algorithms, and Complexity Classes for solving computationally hard problems.

|       | <b>TextBooks ( T )</b>                               |               |                |
|-------|------------------------------------------------------|---------------|----------------|
| Sr No | Title                                                | Author        | Publisher Name |
| T-1   | INTRODUCTION TO THE DESIGN AND ANALYSIS OF ALGORITHM | ANANY LEVITIN | PEARSON        |

|       | <b>Reference Books ( R )</b> |                                          |                     |
|-------|------------------------------|------------------------------------------|---------------------|
| Sr No | Title                        | Author                                   | Publisher Name      |
| R-1   | INTRODUCTION TO ALGORITHMS   | C.E. LEISERSON, R.L. RIVEST AND C. STEIN | THOMAS TELFORD LTD. |

| <b>Other Reading ( OR )</b> |                                                                                                                                       |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Sr No                       | Journals articles as Compulsary reading (specific articles, complete reference)                                                       |
| OR-1                        | <a href="https://www.cse.iitd.ac.in/~ssen/col702/notes/root.pdf">https://www.cse.iitd.ac.in/~ssen/col702/notes/root.pdf</a> ,         |
| OR-2                        | <a href="https://cse.iitkgp.ac.in/~pb/algos-lectures-SandeepSen.pdf">https://cse.iitkgp.ac.in/~pb/algos-lectures-SandeepSen.pdf</a> , |

| <b>Relevant Websites ( RW )</b> |                                                                                                                                                                                                                           |                            |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| Sr No                           | (Web address) (only if relevant to the course)                                                                                                                                                                            | Salient Features           |
| RW-1                            | <a href="https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/pages/lecture-notes/">https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/pages/lecture-notes/</a> | Algorithm Content Resource |

An instruction plan is only a tentative plan. The teacher may make some changes in his/her teaching plan. The students are advised to use syllabus for preparation of all examinations. The students are expected to keep themselves updated on the contemporary issues related to the course. Upto 20% of the questions in any examination/Academic tasks can be asked from such issues even if not explicitly mentioned in the instruction plan.

|      |                                                                                                                                                                                                       |                                                     |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| RW-2 | <a href="https://ece.uwaterloo.ca/~dwharder/aads/Lecture_materials/">https://ece.uwaterloo.ca/~dwharder/aads/Lecture_materials/</a>                                                                   | Algorithm Content Resource                          |
| RW-3 | <a href="https://www.programiz.com/dsa">https://www.programiz.com/dsa</a>                                                                                                                             | Beginner-friendly DAA explanations                  |
| RW-4 | <a href="https://cp-algorithms.com/">https://cp-algorithms.com/</a>                                                                                                                                   | Advanced algorithms with proofs and implementations |
| RW-5 | <a href="https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/pages/lecture-notes/">https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/pages/lecture-notes/</a> | Algorithm Content Resource                          |
| RW-6 | <a href="https://jeffe.cs.illinois.edu/teaching/algorithms/">https://jeffe.cs.illinois.edu/teaching/algorithms/</a>                                                                                   | Comprehensive Algorithm Reference                   |

| Virtual Labs ( VL ) |                                                                                         |                                                                                                            |
|---------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Sr No               | (VL) (only if relevant to the course)                                                   | Salient Features                                                                                           |
| VL-1                | <a href="https://visualgo.net/en">https://visualgo.net/en</a>                           | Interactive visualization software for learning and demonstrating algorithms with step-by-step animations. |
| VL-2                | <a href="https://visualizedsa.com/visualizers">https://visualizedsa.com/visualizers</a> | Interactive visualization software for learning and demonstrating algorithms with step-by-step animations. |

| LTP week distribution: (LTP Weeks) |  |
|------------------------------------|--|
| Weeks before MTE                   |  |
| Weeks After MTE                    |  |
| Spill Over (Lecture)               |  |

### Detailed Plan For Lectures

| Week Number | Lecture Number | Broad Topic(Sub Topic)                                                    | Chapters/Sections of Text/reference books | Other Readings, Relevant Websites, Audio Visual Aids, software and Virtual Labs | Lecture Description                                                                                                                                                                                            | Learning Outcomes                                                                                                                                                                          | Pedagogical Tool Demonstration/ Case Study / Images / animation / ppt etc. Planned                                                     | Live Examples                                                                                                                     |
|-------------|----------------|---------------------------------------------------------------------------|-------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Week 1      | Lecture 1      | Analysis of Algorithms and Complexity Measures(Time and Space Complexity) | T-1<br>R-1                                | OR-2<br>VL-1<br>VL-2                                                            | Introduce algorithms, the need for algorithm analysis, time and space complexity, best, average and worst-case analysis, and asymptotic notations (Big-O, Big-O and Big-T) for comparing algorithm efficiency. | Explain the importance of algorithm analysis, distinguish between time and space complexity, evaluate best, average and worst cases, and apply asymptotic notations to compare algorithms. | Interactive lectures, PPT presentations, board illustrations, classroom discussions, numerical examples, and problem-solving sessions. | Compare Linear Search and Binary Search; analyze O(1), O(log n), O(n), and O(n <sup>2</sup> ) complexities using simple programs. |

An instruction plan is only a tentative plan. The teacher may make some changes in his/her teaching plan. The students are advised to use syllabus for preparation of all examinations. The students are expected to keep themselves updated on the contemporary issues related to the course. Upto 20% of the questions in any examination/Academic tasks can be asked from such issues even if not explicitly mentioned in the instruction plan.



|        |           |                                                                                                                                                 |            |                      |                                                                                                                                                                                                                |                                                                                                                                                                                            |                                                                                                                                        |                                                                                                                                   |
|--------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------|------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Week 1 | Lecture 2 | Analysis of Algorithms and Complexity Measures(Time and Space Complexity)                                                                       | T-1<br>R-1 | OR-2<br>VL-1<br>VL-2 | Introduce algorithms, the need for algorithm analysis, time and space complexity, best, average and worst-case analysis, and asymptotic notations (Big-O, Big-O and Big-T) for comparing algorithm efficiency. | Explain the importance of algorithm analysis, distinguish between time and space complexity, evaluate best, average and worst cases, and apply asymptotic notations to compare algorithms. | Interactive lectures, PPT presentations, board illustrations, classroom discussions, numerical examples, and problem-solving sessions. | Compare Linear Search and Binary Search; analyze $O(1)$ , $O(\log n)$ , $O(n)$ , and $O(n^2)$ complexities using simple programs. |
|        | Lecture 3 | Analysis of Algorithms and Complexity Measures (Complexity Analysis of Insertion Sort and Merge Sort)                                           | T-1<br>R-1 | OR-1<br>VL-1<br>VL-2 | Explain the working and complexity analysis of Insertion Sort and Merge Sort and compare their efficiency under different input conditions.                                                                    | Analyze and compare the performance of Insertion Sort and Merge Sort using time complexity concepts.                                                                                       | Algorithm dry run, animations, comparison tables, board work, and problem-solving activities.                                          | Sort student marks using Insertion Sort and compare with Merge Sort for large datasets.                                           |
| Week 2 | Lecture 4 | Analysis of Algorithms and Complexity Measures (Complexity Analysis of Insertion Sort and Merge Sort)                                           | T-1<br>R-1 | OR-1<br>VL-1<br>VL-2 | Explain the working and complexity analysis of Insertion Sort and Merge Sort and compare their efficiency under different input conditions.                                                                    | Analyze and compare the performance of Insertion Sort and Merge Sort using time complexity concepts.                                                                                       | Algorithm dry run, animations, comparison tables, board work, and problem-solving activities.                                          | Sort student marks using Insertion Sort and compare with Merge Sort for large datasets.                                           |
|        | Lecture 5 | Analysis of Algorithms and Complexity Measures (Analysis of Iterative Algorithms)                                                               | T-1<br>R-1 | RW-1<br>VL-1<br>VL-2 | Demonstrate techniques to analyze algorithms containing single, nested, and multiple loops to determine their time complexity.                                                                                 | Compute the time complexity of iterative algorithms using loop analysis.                                                                                                                   | Interactive lecture, coding demonstrations, board illustrations, and numerical examples.                                               | Complexity analysis of finding maximum, array traversal, and nested loop programs.                                                |
|        | Lecture 6 | Analysis of Algorithms and Complexity Measures (Analysis of Recursive Algorithms: Substitution Method, Recursion Tree Method and Master Method) | T-1<br>R-1 | VL-1<br>VL-2         | Introduce recurrence relations and explain the Substitution Method, Recursion Tree Method, and Master Method for analyzing recursive algorithms.                                                               | Form recurrence relations and solve them using different analytical techniques.                                                                                                            | Board illustrations, worked examples, derivations, PPT presentations, and classroom problem solving.                                   | Analyze Merge Sort, Binary Search, and Divide-and-Conquer recurrence relations.                                                   |
| Week 3 | Lecture 7 | Analysis of Algorithms and Complexity Measures (Analysis of Recursive Algorithms: Substitution Method, Recursion Tree Method and Master Method) | T-1<br>R-1 | VL-1<br>VL-2         | Introduce recurrence relations and explain the Substitution Method, Recursion Tree Method, and Master Method for analyzing recursive algorithms.                                                               | Form recurrence relations and solve them using different analytical techniques.                                                                                                            | Board illustrations, worked examples, derivations, PPT presentations, and classroom problem solving.                                   | Analyze Merge Sort, Binary Search, and Divide-and-Conquer recurrence relations.                                                   |



|        |            |                                                                                                          |            |                      |                                                                                                                                                                        |                                                                                                                          |                                                                                 |                                                                                |
|--------|------------|----------------------------------------------------------------------------------------------------------|------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Week 3 | Lecture 8  | Divide-and-Conquer Technique(Strassen's Matrix Multiplication)                                           | T-1<br>R-1 | RW-2<br>VL-1<br>VL-2 | Introduce the Divide-and-Conquer paradigm and explain Strassen's Matrix Multiplication algorithm for efficient matrix multiplication.                                  | Explain the Divide-and-Conquer strategy and analyze Strassen's algorithm.                                                | PPT presentation, numerical demonstration, animations, and board illustrations. | Perform multiplication of 2×2 matrices using Strassen's algorithm.             |
|        | Lecture 9  | Divide-and-Conquer Technique(Order Statistics: Quick select, k-th Smallest and k-th Largest Element)     | T-1<br>R-1 | RW-3<br>VL-1         | Explain Order Statistics and Quick Select algorithm to efficiently determine k-th smallest and largest elements.                                                       | Solve selection problems using Divide-and-Conquer techniques.                                                            | Dry runs, algorithm visualization, problem-solving sessions.                    | Dry runs, algorithm visualization, problem-solving sessions.                   |
| Week 4 | Lecture 10 | String Matching Algorithms and Computational Geometry(Sequential Search and Brute-Force String Matching) | T-1        | RW-5<br>VL-2         | Introduce sequential search and brute-force string matching techniques. Explain hashing-based string matching using the Rabin-Karp algorithm.                          | Implementation of basic string searching algorithms. Apply hashing techniques for efficient pattern matching.            | Coding demonstrations, dry runs, classroom discussion.                          | Search a word in a text document.                                              |
|        |            | String Matching Algorithms and Computational Geometry(Naive Pattern Matching)                            | T-1        | RW-4<br>VL-2         | Introduce sequential search and brute-force string matching techniques. Explain hashing-based string matching using the Rabin-Karp algorithm.                          | Implementation of basic string searching algorithms. Apply hashing techniques for efficient pattern matching.            | Coding demonstrations, dry runs, classroom discussion.                          | Search a word in a text document.                                              |
|        |            | String Matching Algorithms and Computational Geometry(Rabin-Karp Algorithm)                              | T-1        | RW-6<br>VL-1         | Introduce sequential search and brute-force string matching techniques. Explain hashing-based string matching using the Rabin-Karp algorithm.                          | Implementation of basic string searching algorithms. Apply hashing techniques for efficient pattern matching.            | Coding demonstrations, dry runs, classroom discussion.                          | Search a word in a text document.                                              |
|        | Lecture 11 | String Matching Algorithms and Computational Geometry(Knuth-Morris-Pratt Algorithm)                      | T-1<br>R-1 | OR-1<br>VL-1         | Explain the KMP algorithm and construction of the LPS array for efficient pattern matching and Trie data structure with its operations for efficient prefix searching. | Construct the LPS array and perform efficient string matching. Alongwith, insertion and searching operations using Trie. | Diagrammatic explanation, coding demonstration.                                 | Text editor "Find" operation. Google Autocomplete and dictionary applications. |



|        |            |                                                                                                                  |            |                      |                                                                                                                                                                                   |                                                                                                                          |                                                                                                                                    |                                                                                |
|--------|------------|------------------------------------------------------------------------------------------------------------------|------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Week 4 | Lecture 11 | String Matching Algorithms and Computational Geometry(Data Structures for String Processing: Trie (Prefix Tree)) | T-1<br>R-1 | RW-1<br>VL-1         | Explain the KMP algorithm and construction of the LPS array for efficient pattern matching and Trie data structure with its operations for efficient prefix searching.            | Construct the LPS array and perform efficient string matching. Alongwith, insertion and searching operations using Trie. | Diagrammatic explanation, coding demonstration.                                                                                    | Text editor "Find" operation. Google Autocomplete and dictionary applications. |
|        | Lecture 12 | String Matching Algorithms and Computational Geometry(Computational Geometry: Closest-Pair Problem)              | T-1<br>R-1 |                      | Introduce computational geometry concepts and explain Closest-Pair and Convex Hull algorithms.                                                                                    | Solve basic computational geometry problems using efficient algorithms.                                                  | Visual demonstrations, PPT, board illustrations.                                                                                   | GPS navigation, geographical boundary detection.                               |
|        |            | String Matching Algorithms and Computational Geometry(Convex Hull)                                               | T-1<br>R-1 |                      | Introduce computational geometry concepts and explain Closest-Pair and Convex Hull algorithms.                                                                                    | Solve basic computational geometry problems using efficient algorithms.                                                  | Visual demonstrations, PPT, board illustrations.                                                                                   | GPS navigation, geographical boundary detection.                               |
| Week 5 | Lecture 13 | Dynamic Programming (Introduction to Dynamic Programming)                                                        | T-1<br>R-1 | OR-1<br>OR-2<br>VL-1 | Introduce Dynamic Programming, explain overlapping subproblems and optimal substructure, and demonstrate memoization (memory functions) through binomial coefficient computation. | Explain the principles of Dynamic Programming and apply memoization techniques to solve recursive optimization problems. | Interactive lecture, PPT presentation, board illustrations, algorithm visualization, numerical examples, and classroom discussion. | Fibonacci sequence, Binomial Coefficient using recursion and memoization.      |
|        |            | Dynamic Programming (Computing a Binomial Coefficient)                                                           | T-1<br>R-1 | RW-4                 | Introduce Dynamic Programming, explain overlapping subproblems and optimal substructure, and demonstrate memoization (memory functions) through binomial coefficient computation. | Explain the principles of Dynamic Programming and apply memoization techniques to solve recursive optimization problems. | Interactive lecture, PPT presentation, board illustrations, algorithm visualization, numerical examples, and classroom discussion. | Fibonacci sequence, Binomial Coefficient using recursion and memoization.      |





|        |            |                                                           |            |                              |                                                                                                                                                                                   |                                                                                                                          |                                                                                                                                    |                                                                           |
|--------|------------|-----------------------------------------------------------|------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Week 5 | Lecture 13 | Dynamic Programming (Memory Functions)                    | T-1<br>R-1 | OR-1<br>OR-2<br>RW-1<br>RW-2 | Introduce Dynamic Programming, explain overlapping subproblems and optimal substructure, and demonstrate memoization (memory functions) through binomial coefficient computation. | Explain the principles of Dynamic Programming and apply memoization techniques to solve recursive optimization problems. | Interactive lecture, PPT presentation, board illustrations, algorithm visualization, numerical examples, and classroom discussion. | Fibonacci sequence, Binomial Coefficient using recursion and memoization. |
|        | Lecture 14 | Dynamic Programming (Introduction to Dynamic Programming) | T-1<br>R-1 | OR-1<br>OR-2<br>VL-1         | Introduce Dynamic Programming, explain overlapping subproblems and optimal substructure, and demonstrate memoization (memory functions) through binomial coefficient computation. | Explain the principles of Dynamic Programming and apply memoization techniques to solve recursive optimization problems. | Interactive lecture, PPT presentation, board illustrations, algorithm visualization, numerical examples, and classroom discussion. | Fibonacci sequence, Binomial Coefficient using recursion and memoization. |
|        |            | Dynamic Programming (Computing a Binomial Coefficient)    | T-1<br>R-1 | RW-4                         | Introduce Dynamic Programming, explain overlapping subproblems and optimal substructure, and demonstrate memoization (memory functions) through binomial coefficient computation. | Explain the principles of Dynamic Programming and apply memoization techniques to solve recursive optimization problems. | Interactive lecture, PPT presentation, board illustrations, algorithm visualization, numerical examples, and classroom discussion. | Fibonacci sequence, Binomial Coefficient using recursion and memoization. |
|        |            | Dynamic Programming (Memory Functions)                    | T-1<br>R-1 | OR-1<br>OR-2<br>RW-1<br>RW-2 | Introduce Dynamic Programming, explain overlapping subproblems and optimal substructure, and demonstrate memoization (memory functions) through binomial coefficient computation. | Explain the principles of Dynamic Programming and apply memoization techniques to solve recursive optimization problems. | Interactive lecture, PPT presentation, board illustrations, algorithm visualization, numerical examples, and classroom discussion. | Fibonacci sequence, Binomial Coefficient using recursion and memoization. |
|        | Lecture 15 |                                                           |            |                              | Test 1                                                                                                                                                                            |                                                                                                                          |                                                                                                                                    |                                                                           |
| Week 6 | Lecture 16 | Dynamic Programming (Knapsack Problem)                    | T-1<br>R-1 | OR-1<br>OR-2                 | Explain Dynamic Programming formulations for optimization problems including 0/1 Knapsack and Matrix Chain Multiplication.                                                        | Formulate recurrence relations and apply Dynamic Programming to solve optimization problems efficiently.                 | Problem-solving sessions, board work, algorithm dry run, visualization, and classroom discussion.                                  | Resource allocation (Knapsack), Optimal matrix multiplication order.      |



|        |            |                                                                    |            |                      |                                                                                                                                       |                                                                                                          |                                                                                                                     |                                                                      |
|--------|------------|--------------------------------------------------------------------|------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Week 6 | Lecture 17 | Dynamic Programming (Matrix-Chain Multiplication)                  | T-1<br>R-1 | RW-1<br>VL-1         | Explain Dynamic Programming formulations for optimization problems including 0/1 Knapsack and Matrix Chain Multiplication.            | Formulate recurrence relations and apply Dynamic Programming to solve optimization problems efficiently. | Problem-solving sessions, board work, algorithm dry run, visualization, and classroom discussion.                   | Resource allocation (Knapsack), Optimal matrix multiplication order. |
|        | Lecture 18 | Dynamic Programming (Longest Common Subsequence)                   | T-1<br>R-1 | VL-1                 | Explain Dynamic Programming techniques for sequence comparison and optimal search tree construction using recurrence-based solutions. | Apply Dynamic Programming to solve sequence alignment and optimization problems.                         | Interactive lecture, worked examples, algorithm visualization, board illustrations, and problem-solving activities. | DNA sequence comparison, Dictionary search optimization.             |
|        |            | Dynamic Programming (Optimal Binary Search Trees)                  | T-1<br>R-1 | RW-4                 | Explain Dynamic Programming techniques for sequence comparison and optimal search tree construction using recurrence-based solutions. | Apply Dynamic Programming to solve sequence alignment and optimization problems.                         | Interactive lecture, worked examples, algorithm visualization, board illustrations, and problem-solving activities. | DNA sequence comparison, Dictionary search optimization.             |
| Week 7 | Lecture 19 | Dynamic Programming (Longest Common Subsequence)                   | T-1<br>R-1 | VL-1                 | Explain Dynamic Programming techniques for sequence comparison and optimal search tree construction using recurrence-based solutions. | Apply Dynamic Programming to solve sequence alignment and optimization problems.                         | Interactive lecture, worked examples, algorithm visualization, board illustrations, and problem-solving activities. | DNA sequence comparison, Dictionary search optimization.             |
|        |            | Dynamic Programming (Optimal Binary Search Trees)                  | T-1<br>R-1 | RW-4                 | Explain Dynamic Programming techniques for sequence comparison and optimal search tree construction using recurrence-based solutions. | Apply Dynamic Programming to solve sequence alignment and optimization problems.                         | Interactive lecture, worked examples, algorithm visualization, board illustrations, and problem-solving activities. | DNA sequence comparison, Dictionary search optimization.             |
|        |            | <b>SPILL OVER</b>                                                  |            |                      |                                                                                                                                       |                                                                                                          |                                                                                                                     |                                                                      |
| Week 7 | Lecture 20 |                                                                    |            |                      | Spill Over                                                                                                                            |                                                                                                          |                                                                                                                     |                                                                      |
|        | Lecture 21 |                                                                    |            |                      | Spill Over                                                                                                                            |                                                                                                          |                                                                                                                     |                                                                      |
|        |            | <b>MID-TERM</b>                                                    |            |                      |                                                                                                                                       |                                                                                                          |                                                                                                                     |                                                                      |
| Week 8 | Lecture 22 | Graph Algorithms and Network Optimization (Introduction to Graphs) | T-1<br>R-1 | OR-1<br>VL-1<br>VL-2 | Introduce graph representations and explain BFS traversal for graph exploration.                                                      | Represent graphs and perform BFS traversal to solve graph traversal problems.                            | Interactive lecture, graph visualization, coding demonstration, board illustrations.                                |                                                                      |

An instruction plan is only a tentative plan. The teacher may make some changes in his/her teaching plan. The students are advised to use syllabus for preparation of all examinations. The students are expected to keep themselves updated on the contemporary issues related to the course. Upto 20% of the questions in any examination/Academic tasks can be asked from such issues even if not explicitly mentioned in the instruction plan.



|        |            |                                                                        |            |                      |                                                                                                                                                                                               |                                                                               |                                                                                      |  |
|--------|------------|------------------------------------------------------------------------|------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|--|
| Week 8 | Lecture 22 | Graph Algorithms and Network Optimization (Breadth-First Search (BFS)) | T-1<br>R-1 | RW-3<br>VL-1         | Introduce graph representations and explain BFS traversal for graph exploration.                                                                                                              | Represent graphs and perform BFS traversal to solve graph traversal problems. | Interactive lecture, graph visualization, coding demonstration, board illustrations. |  |
|        | Lecture 23 | Graph Algorithms and Network Optimization (Introduction to Graphs)     | T-1<br>R-1 | OR-1<br>VL-1<br>VL-2 | Introduce graph representations and explain BFS traversal for graph exploration.                                                                                                              | Represent graphs and perform BFS traversal to solve graph traversal problems. | Interactive lecture, graph visualization, coding demonstration, board illustrations. |  |
|        |            | Graph Algorithms and Network Optimization (Breadth-First Search (BFS)) | T-1<br>R-1 | RW-3<br>VL-1         | Introduce graph representations and explain BFS traversal for graph exploration.                                                                                                              | Represent graphs and perform BFS traversal to solve graph traversal problems. | Interactive lecture, graph visualization, coding demonstration, board illustrations. |  |
|        | Lecture 24 | Graph Algorithms and Network Optimization (Depth-First Search (DFS))   | T-1<br>R-1 | RW-3<br>VL-1         | <p>Explain DFS traversal and Topological Sorting for directed acyclic graphs.</p> <p>Activity: Non Proctored code based test. Also used for shortlisting students for fast learner track.</p> | Apply DFS and Topological Sort to solve dependency and traversal problems.    | Algorithm visualization, dry run, classroom discussion, coding demonstration.        |  |
| Week 9 | Lecture 25 | Graph Algorithms and Network Optimization (Topological Sort)           | T-1<br>R-1 | OR-1<br>OR-2<br>VL-1 | Explain DFS traversal and Topological Sorting for directed acyclic graphs.                                                                                                                    | Apply DFS and Topological Sort to solve dependency and traversal problems.    | Algorithm visualization, dry run, classroom discussion, coding demonstration.        |  |



|         |            |                                                    |            |                      |                                                                                                      |                                                                                          |                                                                                         |  |
|---------|------------|----------------------------------------------------|------------|----------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|--|
| Week 9  | Lecture 26 | Greedy Technique (Introduction to Greedy approach) | T-1        | OR-1<br>OR-2<br>RW-2 | Explain greedy strategy and apply it to optimization problems including Fractional Knapsack and MST. | Identify greedy-choice property and solve optimization problems using greedy techniques. | Interactive lecture, algorithm visualization, numerical examples, coding demonstration. |  |
|         |            | Greedy Technique (Knapsack Problem)                | T-1        | OR-1<br>OR-2<br>VL-2 | Explain greedy strategy and apply it to optimization problems including Fractional Knapsack and MST. | Identify greedy-choice property and solve optimization problems using greedy techniques. | Interactive lecture, algorithm visualization, numerical examples, coding demonstration. |  |
|         |            | Greedy Technique (Minimum Spanning Trees)          | T-1        | RW-1<br>VL-2         | Explain greedy strategy and apply it to optimization problems including Fractional Knapsack and MST. | Identify greedy-choice property and solve optimization problems using greedy techniques. | Interactive lecture, algorithm visualization, numerical examples, coding demonstration. |  |
|         | Lecture 27 | Greedy Technique (Introduction to Greedy approach) | T-1        | OR-1<br>OR-2<br>RW-2 | Explain greedy strategy and apply it to optimization problems including Fractional Knapsack and MST. | Identify greedy-choice property and solve optimization problems using greedy techniques. | Interactive lecture, algorithm visualization, numerical examples, coding demonstration. |  |
|         |            | Greedy Technique (Knapsack Problem)                | T-1        | OR-1<br>OR-2<br>VL-2 | Explain greedy strategy and apply it to optimization problems including Fractional Knapsack and MST. | Identify greedy-choice property and solve optimization problems using greedy techniques. | Interactive lecture, algorithm visualization, numerical examples, coding demonstration. |  |
|         |            | Greedy Technique (Minimum Spanning Trees)          | T-1        | RW-1<br>VL-2         | Explain greedy strategy and apply it to optimization problems including Fractional Knapsack and MST. | Identify greedy-choice property and solve optimization problems using greedy techniques. | Interactive lecture, algorithm visualization, numerical examples, coding demonstration. |  |
| Week 10 | Lecture 28 | Greedy Technique(Prim's Algorithm)                 | T-1<br>R-1 | RW-1<br>VL-1         | Explain construction of Minimum Spanning Trees using Prim's and Kruskal's algorithms.                | Apply MST algorithms and compare their performance.                                      | Board illustrations, graph visualization, problem-solving session.                      |  |





|         |            |                                                         |            |                      |                                                                                                                                         |                                                                                                                                     |                                                                                                                |  |
|---------|------------|---------------------------------------------------------|------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|--|
| Week 10 | Lecture 28 | Greedy Technique(Kruskal's Algorithm)                   | T-1<br>R-1 | RW-1<br>VL-2         | Explain construction of Minimum Spanning Trees using Prim's and Kruskal's algorithms.                                                   | Apply MST algorithms and compare their performance.                                                                                 | Board illustrations, graph visualization, problem-solving session.                                             |  |
|         | Lecture 29 | Greedy Technique(Single-Source Shortest Paths)          | T-1<br>R-1 | OR-1<br>VL-1<br>VL-2 | Explain shortest path algorithms and their applications in weighted graphs.                                                             | Solve shortest path problems using appropriate graph algorithms.                                                                    | Interactive lecture, graph simulation, coding demonstration.                                                   |  |
|         |            | Greedy Technique(All-Pairs Shortest Paths)              | T-1<br>R-1 | OR-1<br>OR-2<br>VL-2 | Explain shortest path algorithms and their applications in weighted graphs.                                                             | Solve shortest path problems using appropriate graph algorithms.                                                                    | Interactive lecture, graph simulation, coding demonstration.                                                   |  |
|         | Lecture 30 |                                                         |            |                      | Test 2                                                                                                                                  |                                                                                                                                     |                                                                                                                |  |
| Week 11 | Lecture 31 | Transform-and-Conquer Techniques(Balanced Search Trees) | T-1<br>R-1 | RW-2                 | Explain the Transform-and-Conquer paradigm and demonstrate Balanced Search Trees and Counting Sort for efficient searching and sorting. | Explain the Transform-and-Conquer strategy and apply Balanced Search Trees and Counting Sort to appropriate computational problems. | Interactive lecture, board illustrations, algorithm visualization, coding demonstration, classroom discussion. |  |
|         |            | Transform-and-Conquer Techniques(Counting Sort)         | T-1<br>R-1 | RW-4<br>RW-5<br>VL-1 | Explain the Transform-and-Conquer paradigm and demonstrate Balanced Search Trees and Counting Sort for efficient searching and sorting. | Explain the Transform-and-Conquer strategy and apply Balanced Search Trees and Counting Sort to appropriate computational problems. | Interactive lecture, board illustrations, algorithm visualization, coding demonstration, classroom discussion. |  |
|         | Lecture 32 | Transform-and-Conquer Techniques(Balanced Search Trees) | T-1<br>R-1 | RW-2                 | Explain the Transform-and-Conquer paradigm and demonstrate Balanced Search Trees and Counting Sort for efficient searching and sorting. | Explain the Transform-and-Conquer strategy and apply Balanced Search Trees and Counting Sort to appropriate computational problems. | Interactive lecture, board illustrations, algorithm visualization, coding demonstration, classroom discussion. |  |



|         |            |                                                         |            |                      |                                                                                                                                                        |                                                                                                                                     |                                                                                                                 |                                                                                                      |
|---------|------------|---------------------------------------------------------|------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Week 11 | Lecture 32 | Transform-and-Conquer Techniques(Counting Sort)         | T-1<br>R-1 | RW-4<br>RW-5<br>VL-1 | Explain the Transform-and-Conquer paradigm and demonstrate Balanced Search Trees and Counting Sort for efficient searching and sorting.                | Explain the Transform-and-Conquer strategy and apply Balanced Search Trees and Counting Sort to appropriate computational problems. | Interactive lecture, board illustrations, algorithm visualization, coding demonstration, classroom discussion.  |                                                                                                      |
|         | Lecture 33 | Transform-and-Conquer Techniques(Radix Sort)            | T-1<br>R-1 | RW-3<br>VL-2         | Explain non-comparison-based sorting algorithms and analyze their efficiency and applications.                                                         | Compare Radix Sort and Bucket Sort and select suitable sorting techniques for different datasets.                                   | Live coding, numerical examples, algorithm dry run, problem-solving session.                                    |                                                                                                      |
|         |            | Transform-and-Conquer Techniques(Bucket Sort)           | T-1<br>R-1 | VL-1<br>VL-2         | Explain non-comparison-based sorting algorithms and analyze their efficiency and applications.                                                         | Compare Radix Sort and Bucket Sort and select suitable sorting techniques for different datasets.                                   | Live coding, numerical examples, algorithm dry run, problem-solving session.                                    |                                                                                                      |
| Week 12 | Lecture 34 | Advanced Algorithmic Techniques(Monotonic Stack)        | T-1<br>R-1 |                      | Introduce advanced algorithmic techniques for solving array and sequence problems efficiently using specialized data structures and traversal methods. | Apply Monotonic Stack, Monotonic Queue, and Two Pointers techniques to develop efficient linear-time solutions.                     | Interactive lecture, coding demonstration, visualization of stack and queue operations, guided problem-solving. | Largest Rectangle in Histogram, Sliding Window Maximum, Two Sum/Pair Sum, Container With Most Water. |
|         |            | Advanced Algorithmic Techniques(Monotonic Queue)        | T-1<br>R-1 |                      | Introduce advanced algorithmic techniques for solving array and sequence problems efficiently using specialized data structures and traversal methods. | Apply Monotonic Stack, Monotonic Queue, and Two Pointers techniques to develop efficient linear-time solutions.                     | Interactive lecture, coding demonstration, visualization of stack and queue operations, guided problem-solving. | Largest Rectangle in Histogram, Sliding Window Maximum, Two Sum/Pair Sum, Container With Most Water. |
|         |            | Advanced Algorithmic Techniques(Two pointers Technique) | T-1<br>R-1 | RW-3                 | Introduce advanced algorithmic techniques for solving array and sequence problems efficiently using specialized data structures and traversal methods. | Apply Monotonic Stack, Monotonic Queue, and Two Pointers techniques to develop efficient linear-time solutions.                     | Interactive lecture, coding demonstration, visualization of stack and queue operations, guided problem-solving. |                                                                                                      |



|         |            |                                                                          |            |              |                                                                                                                                                        |                                                                                                                        |                                                                                                                             |                                                                                                      |
|---------|------------|--------------------------------------------------------------------------|------------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Week 12 | Lecture 35 | Advanced Algorithmic Techniques(Monotonic Stack)                         | T-1<br>R-1 |              | Introduce advanced algorithmic techniques for solving array and sequence problems efficiently using specialized data structures and traversal methods. | Apply Monotonic Stack, Monotonic Queue, and Two Pointers techniques to develop efficient linear-time solutions.        | Interactive lecture, coding demonstration, visualization of stack and queue operations, guided problem-solving.             | Largest Rectangle in Histogram, Sliding Window Maximum, Two Sum/Pair Sum, Container With Most Water. |
|         |            | Advanced Algorithmic Techniques(Monotonic Queue)                         | T-1<br>R-1 |              | Introduce advanced algorithmic techniques for solving array and sequence problems efficiently using specialized data structures and traversal methods. | Apply Monotonic Stack, Monotonic Queue, and Two Pointers techniques to develop efficient linear-time solutions.        | Interactive lecture, coding demonstration, visualization of stack and queue operations, guided problem-solving.             | Largest Rectangle in Histogram, Sliding Window Maximum, Two Sum/Pair Sum, Container With Most Water. |
|         |            | Advanced Algorithmic Techniques(Two pointers Technique)                  | T-1<br>R-1 | RW-3         | Introduce advanced algorithmic techniques for solving array and sequence problems efficiently using specialized data structures and traversal methods. | Apply Monotonic Stack, Monotonic Queue, and Two Pointers techniques to develop efficient linear-time solutions.        | Interactive lecture, coding demonstration, visualization of stack and queue operations, guided problem-solving.             |                                                                                                      |
|         | Lecture 36 |                                                                          |            |              | Test - Code based                                                                                                                                      |                                                                                                                        |                                                                                                                             |                                                                                                      |
| Week 13 | Lecture 37 | Backtracking, Approximation and Complexity Classes(n-Queens Problem)     | T-1<br>R-1 | RW-6         | Introduce the Backtracking paradigm and demonstrate its application in solving combinatorial and constraint satisfaction problems.                     | Apply backtracking techniques to systematically explore solution spaces and solve combinatorial optimization problems. | Interactive lecture, state-space tree visualization, recursion tree analysis, coding demonstration, guided problem-solving. |                                                                                                      |
|         |            | Backtracking, Approximation and Complexity Classes (Hamiltonian Circuit) | T-1<br>R-1 | RW-2<br>VL-2 | Introduce the Backtracking paradigm and demonstrate its application in solving combinatorial and constraint satisfaction problems.                     | Apply backtracking techniques to systematically explore solution spaces and solve combinatorial optimization problems. | Interactive lecture, state-space tree visualization, recursion tree analysis, coding demonstration, guided problem-solving. |                                                                                                      |



|         |            |                                                                                             |            |              |                                                                                                                                    |                                                                                                                        |                                                                                                                             |                                                                      |
|---------|------------|---------------------------------------------------------------------------------------------|------------|--------------|------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Week 13 | Lecture 37 | Backtracking, Approximation and Complexity Classes(Subset-Sum Problem)                      | T-1<br>R-1 | VL-1<br>VL-2 | Introduce the Backtracking paradigm and demonstrate its application in solving combinatorial and constraint satisfaction problems. | Apply backtracking techniques to systematically explore solution spaces and solve combinatorial optimization problems. | Interactive lecture, state-space tree visualization, recursion tree analysis, coding demonstration, guided problem-solving. |                                                                      |
|         | Lecture 38 | Backtracking, Approximation and Complexity Classes(Branch and Bound: Assignment Problem)    | T-1<br>R-1 |              | Explain the Branch and Bound technique for solving optimization problems by systematically pruning the search space.               | Apply Branch and Bound to solve optimization problems efficiently and compare it with Backtracking.                    | Algorithm visualization, state-space tree construction, numerical examples, classroom discussion.                           | Assignment Problem, 0/1 Knapsack, Traveling Salesman Problem (TSP).  |
|         |            | Backtracking, Approximation and Complexity Classes (Knapsack Problem)                       | T-1<br>R-1 |              | Explain the Branch and Bound technique for solving optimization problems by systematically pruning the search space.               | Apply Branch and Bound to solve optimization problems efficiently and compare it with Backtracking.                    | Algorithm visualization, state-space tree construction, numerical examples, classroom discussion.                           | Assignment Problem, 0/1 Knapsack, Traveling Salesman Problem (TSP).  |
|         |            | Backtracking, Approximation and Complexity Classes (Traveling Salesman Problem)             | T-1<br>R-1 |              | Explain the Branch and Bound technique for solving optimization problems by systematically pruning the search space.               | Apply Branch and Bound to solve optimization problems efficiently and compare it with Backtracking.                    | Algorithm visualization, state-space tree construction, numerical examples, classroom discussion.                           | Assignment Problem, 0/1 Knapsack, Traveling Salesman Problem (TSP).  |
|         | Lecture 39 | Backtracking, Approximation and Complexity Classes (Approximation Algorithms: Vertex-Cover) | T-1<br>R-1 |              | Introduce approximation algorithms for NP-hard optimization problems and analyze their approximation strategies and performance.   | Apply approximation algorithms to obtain near-optimal solutions for computationally hard problems                      | Interactive lecture, algorithm comparison, case studies, problem-solving activities.                                        | Vertex Cover, Set Cover, First Fit Decreasing (FFD) for Bin Packing. |
|         |            | Backtracking, Approximation and Complexity Classes(Set-Covering)                            | T-1<br>R-1 |              | Introduce approximation algorithms for NP-hard optimization problems and analyze their approximation strategies and performance.   | Apply approximation algorithms to obtain near-optimal solutions for computationally hard problems                      | Interactive lecture, algorithm comparison, case studies, problem-solving activities.                                        | Vertex Cover, Set Cover, First Fit Decreasing (FFD) for Bin Packing. |





|         |            |                                                                                                                           |            |                      |                                                                                                                                                                                                                        |                                                                                                                                       |                                                                                                  |                                                                      |
|---------|------------|---------------------------------------------------------------------------------------------------------------------------|------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Week 13 | Lecture 39 | Backtracking, Approximation and Complexity Classes(Bin Packing Problems)                                                  | T-1<br>R-1 |                      | Introduce approximation algorithms for NP-hard optimization problems and analyze their approximation strategies and performance.                                                                                       | Apply approximation algorithms to obtain near-optimal solutions for computationally hard problems                                     | Interactive lecture, algorithm comparison, case studies, problem-solving activities.             | Vertex Cover, Set Cover, First Fit Decreasing (FFD) for Bin Packing. |
| Week 14 | Lecture 40 | Backtracking, Approximation and Complexity Classes (Complexity Classes: Concepts of P, NP, NP-Hard, NP-Complete Problems) | T-1<br>R-1 | RW-4<br>RW-5<br>RW-6 | Introduce computational complexity theory and explain the classification of computational problems based on solvability and computational difficulty. Summarize the major algorithmic paradigms covered in the course. | Differentiate between P, NP, NP-Hard, and NP-Complete problems and evaluate algorithmic approaches based on computational complexity. | Interactive lecture, concept mapping, comparative analysis, classroom discussion, recap session. |                                                                      |

## SPILL OVER

|         |            |  |  |  |            |  |  |  |
|---------|------------|--|--|--|------------|--|--|--|
| Week 14 | Lecture 41 |  |  |  | Spill Over |  |  |  |
|         | Lecture 42 |  |  |  | Spill Over |  |  |  |
| Week 15 | Lecture 43 |  |  |  | Spill Over |  |  |  |
|         | Lecture 44 |  |  |  | Spill Over |  |  |  |
|         | Lecture 45 |  |  |  | Spill Over |  |  |  |

### Scheme for CA:

CA Category of this Course Code is:A0203 (2 best out of 3)

| Component         | Weightage (%) | Mapped CO(s)            |
|-------------------|---------------|-------------------------|
| Test 1            | 50            | CO1, CO2                |
| Test 2            | 50            | CO3, CO4                |
| Test - Code based | 50            | CO1, CO2, CO3, CO4, CO5 |

### Details of Academic Task(s)

An instruction plan is only a tentative plan. The teacher may make some changes in his/her teaching plan. The students are advised to use syllabus for preparation of all examinations. The students are expected to keep themselves updated on the contemporary issues related to the course. Upto 20% of the questions in any examination/Academic tasks can be asked from such issues even if not explicitly mentioned in the instruction plan.



| Academic Task     | Objective                                                                                                                                                                 | Detail of Academic Task                                                                                                                                                         | Nature of Academic Task<br>(group/individuals) | Academic Task Mode | Marks | Allotment / submission Week |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|--------------------|-------|-----------------------------|
| Test 1            | Assess students' conceptual understanding, analytical thinking, and problem-solving skills related to algorithmic concepts through objective and coding-based assessment. | Individual assessment comprising 15 MCQs to evaluate conceptual understanding and programming skills.                                                                           | Individual                                     | Offline            | 30    | 4 / 5                       |
| Test 2            | Assess students' conceptual understanding, analytical thinking, and problem-solving skills related to algorithmic concepts through objective and coding-based assessment. | Individual assessment comprising 15 MCQs to evaluate conceptual understanding and programming skills.                                                                           | Individual                                     | Offline            | 30    | 9 / 10                      |
| Test - Code based | Assess students' programming proficiency by designing, implementing, and testing efficient algorithmic solutions in a practical environment.                              | Individual BYOD-based practical coding assessment comprising MCQs and 2 programming problems requiring students to design, implement, test, and optimize algorithmic solutions. | Individual                                     | Online             | 30    | 12 / 13                     |

### Detailed Plan For Practicals

| Practical No | Broad topic       | Subtopic                                                               | Other Readings | Learning Outcomes                                                                                              |
|--------------|-------------------|------------------------------------------------------------------------|----------------|----------------------------------------------------------------------------------------------------------------|
| Practical 1  | List Of Practical | 1. Implement Insertion Sort algorithm and analyze its time complexity. | VL-1<br>VL-2   | Analyze and implement the Insertion Sort algorithm and evaluate its time complexity for different input cases. |
| Practical 2  | List Of Practical | 2. Implement Merge Sort algorithm and analyze its time complexity.     | VL-1<br>VL-2   | Implement the Divide-and-Conquer based Merge Sort algorithm and analyze its efficiency.                        |

An instruction plan is only a tentative plan. The teacher may make some changes in his/her teaching plan. The students are advised to use syllabus for preparation of all examinations. The students are expected to keep themselves updated on the contemporary issues related to the course. Upto 20% of the questions in any examination/Academic tasks can be asked from such issues even if not explicitly mentioned in the instruction plan.

|              |                   |                                                                                                  |              |                                                                                                                         |
|--------------|-------------------|--------------------------------------------------------------------------------------------------|--------------|-------------------------------------------------------------------------------------------------------------------------|
| Practical 3  | List Of Practical | 3. Implement Strassen's Matrix Multiplication algorithm.                                         | VL-1<br>VL-2 | Implement Strassen's algorithm to perform efficient matrix multiplication and compare it with the conventional approach |
| Practical 4  | List Of Practical | 4. Implement the Quick Select algorithm to find the k-th smallest and k-th largest element.      | VL-1<br>VL-2 | Apply the Quick Select algorithm to efficiently find the k-th smallest and k-th largest elements.                       |
| Practical 5  | List Of Practical | 5. Implement the Naïve String Matching algorithm.                                                | VL-1<br>VL-2 | Implement the Naïve String Matching algorithm and evaluate its pattern matching performance.                            |
| Practical 6  | List Of Practical | 6. Implement the Rabin-Karp algorithm for string matching using hashing.                         | VL-1<br>VL-2 | Implement the Rabin-Karp algorithm using hashing for efficient string pattern matching.                                 |
| Practical 7  | List Of Practical | 7. Implement the Knuth-Morris-Pratt (KMP) algorithm for efficient string matching.               | VL-1<br>VL-2 | Develop the KMP algorithm to perform optimized string matching using the LPS array.                                     |
| Practical 8  | List Of Practical | 8. Implement a Trie (Prefix Tree) for insertion and searching of strings.                        | VL-1<br>VL-2 | Implement a Trie data structure to support efficient insertion and searching of strings.                                |
| Practical 9  | List Of Practical | 9. Implement the Convex Hull algorithm using Graham Scan                                         | VL-1<br>VL-2 | Implement the Graham Scan algorithm to compute the convex hull of a given set of points.                                |
| Practical 10 | List Of Practical | 10. Implement the Longest Common Subsequence (LCS) algorithm using Dynamic Programming.          | VL-1<br>VL-2 | Apply Dynamic Programming to determine the Longest Common Subsequence between two strings.                              |
| Practical 11 | List Of Practical | 11. Implement Matrix Chain Multiplication using Dynamic Programming.                             | VL-1<br>VL-2 | Implement Dynamic Programming to optimize the order of matrix multiplications.                                          |
| Practical 12 | List Of Practical | 12. Implement the 0/1 Knapsack problem using Dynamic Programming.                                | VL-1<br>VL-2 | Solve the 0/1 Knapsack optimization problem using Dynamic Programming.                                                  |
| Practical 13 | List Of Practical | 13. Implement Breadth-First Search (BFS) and Depth-First Search (DFS) graph traversal algorithms | VL-1<br>VL-2 | Implement Breadth-First Search and Depth-First Search algorithms for graph traversal.                                   |
| Practical 14 | List Of Practical | 14. Implement Prim's algorithm to find the Minimum Spanning Tree of a weighted graph.            | VL-1<br>VL-2 | Implement Prim's algorithm to construct the Minimum Spanning Tree of a weighted graph.                                  |
| Practical 15 | List Of Practical | 15. Implement Kruskal's algorithm to find the Minimum Spanning Tree of a weighted graph.         | VL-1<br>VL-2 | Implement Kruskal's algorithm using the greedy approach to find the Minimum Spanning Tree.                              |
| Practical 16 | List Of Practical | 16. Implement Dijkstra's algorithm to find the Single-Source Shortest Path in a weighted graph   | VL-1<br>VL-2 | Implement Dijkstra's algorithm to compute the shortest paths from a source vertex in a weighted graph.                  |
| Practical 17 | List Of Practical | 17. Implement the Largest Rectangle in Histogram problem using the Monotonic Stack technique.    | VL-1<br>VL-2 | Apply a Monotonic Stack to efficiently determine the largest rectangular area in a histogram.                           |

|              |                   |                                                                                          |              |                                                                                                            |
|--------------|-------------------|------------------------------------------------------------------------------------------|--------------|------------------------------------------------------------------------------------------------------------|
| Practical 18 | List Of Practical | 18. Implement the Longest Palindromic Substring problem using the Two-Pointer technique. | VL-1<br>VL-2 | Implement the Two-Pointer technique to identify the longest palindromic substring efficiently.             |
| Practical 19 | List Of Practical | 19. Implement the N-Queens problem using the Backtracking technique.                     | VL-1<br>VL-2 | Solve the N-Queens problem using the Backtracking technique to generate valid board configurations.        |
| Practical 20 | List Of Practical | 20. Implement the Traveling Salesman Problem using the Branch and Bound technique.       | VL-1<br>VL-2 | Implement the Branch and Bound technique to obtain an optimal solution for the Traveling Salesman Problem. |
|              | <b>SPILL OVER</b> |                                                                                          |              |                                                                                                            |
| Practical 21 | Spill Over        |                                                                                          |              |                                                                                                            |